

PAPER

An exact constraint strategy for solving stiff chemical kinetics based on physics-informed neural network

To cite this article: Hanmin Fang *et al* 2026 *Phys. Scr.* **101** 046003

View the [article online](#) for updates and enhancements.

You may also like

- [First-principles study of the gas-sensing properties of a Cu₂ cluster-decorated CrS₂ monolayer towards harmful gases](#)
Daoguang Liu, Xiang Lin, Hui Liu et al.
- [Physical layer encryption scheme based on human biometric keys and five-dimensional hyperchaotic systems](#)
Fei Li, Hao Jing and Yonggang Su
- [Study on the impact of complexity variation on chaotic system performance and hardware implementation of a weak signal detection system](#)
Shaohui Yan, Zihao Guo and Weitao Hu



PAPER

An exact constraint strategy for solving stiff chemical kinetics based on physics-informed neural network

Hanmin Fang¹, Wenlong Huang^{1,2,*} , Na Wang^{3,*} and Haolong Li⁴ ¹ School of Computer Science and Technology, Anhui University of Technology, Ma'anshan, Anhui 243002, People's Republic of China² Anhui Province Key Laboratory of Digital Twin Technology in Metallurgical Industry, People's Republic of China³ School of Geographical Sciences and Environment, Shijiazhuang University, Shijiazhuang 050035, People's Republic of China⁴ Department of Physics, College of Sciences, Tianjin University of Science and Technology, Tianjin 300457, People's Republic of China

* Authors to whom any correspondence should be addressed.

E-mail: whuang@ahut.edu.cn and wangna_sjzc@sina.com**Keywords:** ordinary differential equations, physics informed neural network, exact conservation, stiff chemical kinetics**Abstract**

Stiff chemical kinetics plays a crucial role in various disciplines, including atmospheric chemistry, environmental engineering, and combustion science. In this work, we focus on the ROBER problem, a typical stiff system characterized by a set of strongly coupled ordinary differential equations (ODEs). The system's stiffness arises from the coexistence of ultrafast reactions and slow dynamics, which poses computational challenges to numerical stability and accuracy. Recently, Physics-Informed Neural Networks (PINN), a deep learning based framework, has been widely applied to solve various partial differential equations. However, the standard PINN faces optimization difficulties in multi-scale, strongly nonlinear systems like the ROBER problem. To address this problem, we propose a new strategy called Exact-Constraint PINN (EC-PINN), which integrates conservation law as exact constraint directly into the loss function, enforcing rigorous conservation throughout the training procedure. Based on this framework, we introduce pure exact conservation approaches (with two formulations differing in the treatment of indirect variables) and a hybrid strategy applying soft constraint on the conservation law during the training procedure. Numerical experiments show that the hybrid approach achieves higher accuracy than pure methods and prior soft constraint approach, crucially preserving the conservation law. We also calculate the correlation coefficients among relative error, loss function and its gradient. Analysis reveals a strong correlation between relative error and loss function, validating loss minimization for solving PDEs. In summary, our EC-PINN presents a novel and effective approach for solving stiff chemical kinetics problems, offering improved computational reliability and physical consistency.

1. Introduction

Stiff chemical kinetics plays a critical role in various scientific fields. For instance, in atmospheric chemistry, it helps analyze the dynamic behaviors of pollutants (e.g., ozone and nitrogen oxides), clarifying their impacts on air quality and climate change [1]. In environmental engineering, it can be used to predict the performance of hydrodynamic cavitation reactors for degrading organic pollutants [2]. This methodology also provides insights into combustion science, particularly in understanding its ability to model multi-scale physical and chemical processes, making it essential for combustion research and industrial applications [3]. Therefore, developing an efficient and accurate solver for stiff chemical kinetics is a central issue across multiple scientific disciplines.

The ROBER problem, a typical model in stiff chemical kinetics, is defined by a set of coupled ordinary differential equations (ODEs) that describe the temporal evolution of species concentrations during chemical reactions [4]. A distinctive characteristic of this system is the coexistence of both ultrafast and slow reaction

rates, differing by several orders of magnitude. This leads to the stiffness of the system. In the last decades, conventional numerical methods have been employed to address the ROBER problem. For instance, in 2001, Voss *et al* employed a Rosenbrock algorithm to address stiff chemical reaction systems and utilized parallel computing to improve computational efficiency [5]. Later, Sandu *et al* implemented an implicit Runge-Kutta solver in Kinetic PreProcessor-2.2 (KPP) for direct decoupled and discrete adjoint sensitivity analysis of stiff chemical systems, and demonstrated its applicability [6]. Recently, Khalsaraei *et al* proposed a class of multi-step multi-derivative hybrid methods and applied these methods to solve the ROBER problem [7]. While these approaches are stable and accurate, they may involve substantial linear operations. In addition, the construction and storage of Jacobian matrices impose substantial computational and memory demands.

Physics-Informed Neural Network (PINN), a deep learning-based solver proposed by Raissi *et al* in 2019, has attracted significant interest across various disciplines [8]. Since it is a mesh free approach, PINN is advantageous in high dimension problem and complex geometry [9, 10]. While it has been successfully applied to various fields, such as fluid mechanics [11, 12], biomedical engineering [13, 14], and materials science [15, 16], PINN may encounter optimization difficulties in multi-scale, strongly nonlinear systems [17, 18], such as the ROBER problem. To address this issue, Ji *et al* proposed Stiff-PINN, a method that mitigates stiffness in ODEs by employing Quasi-Steady-State Assumptions (QSSA) [19]. Meanwhile, Weng *et al* introduced a multiscale approach (MPINN) to improve PINN's performance for stiff chemical kinetics [20]. Recently, Fang *et al* incorporated a soft conservation constraint into PINN (CC-PINN), improving numerical stability for long-term simulations [21]. Nevertheless, these methods [19–23] cannot guarantee that the conservation law in stiff chemical kinetics are satisfied during the training procedure of PINN.

It is believed that exact satisfaction of conservation law is beneficial for achieving long-time simulations. Therefore, we propose an Exact-Constraint Physics-Informed Neural Network (EC-PINN), a novel framework that enforces conservation law as exact constraint through direct integration into the loss function, thereby guaranteeing strict physical consistency throughout the training procedure. Based on this framework, we implement both pure exact conservation (with two formulations) and a hybrid strategy. One pure formulation removes governing equations for indirect variables, applying constraints only to direct ones; the other retains all terms during training, with both formulations computing indirect variables from conserved quantities at the output stage. Superior accuracy is achieved by the hybrid approach, which applies a soft conservation constraint throughout training, demonstrating their significant value for improving PINN's accuracy. Moreover, compared to [21], our EC-PINN enhances overall performance while critically ensuring the satisfaction of the conservation law. To improve the interpretability of PINN, we calculate the correlation coefficients among the relative error, the loss, and its gradient. Analysis shows that there is a high correlation between the relative error and the total loss of s_1 , $\tilde{s}_2$, and s_3 . Therefore, solving PDEs through loss minimization in PINN is reasonable. Generally, our EC-PINN addresses stiff chemical kinetics challenges, offering improved computational reliability and reasonable physical interpretability. In addition, our EC-PINN can be easily extended to more realistic multi-species kinetic systems [19], and could also be used for other simulation issues, such as reactive transport and combustion modeling [24, 25]. It should be emphasized that the proposed EC-PINN is not intended to compete computationally with existing methods [19–21]. Notably, this constraint is fully compatible with existing approaches. In the future, we plan to combine the exact constraint with existing methods, aiming to provide a new approach for long time integration tasks in these fields.

The rest of this work is organized as follows. In section 2, we introduce the framework of standard PINN and the architecture of the shared-branch network. We then propose the specific form and normalization scheme of the ROBER problem, along with the algorithms of our EC-PINN in section 3. In section 4, numerical experiments are presented to demonstrate the performance of our method. Finally, we present a summary and discussion in section 5.

2. Preliminaries

2.1. A brief review of the standard PINN

PINN is a framework which integrates deep learning with physical equations [8, 26]. It enforces partial differential equations (PDEs) and their initial and boundary conditions within the loss function. By minimizing the loss function, one can obtain the solutions of PDEs. In general, linear and nonlinear PDEs can be expressed in the following form [27]:

$$u_t + \mathcal{N}[u; \lambda] = 0, \quad (1)$$

which satisfies the following initial and boundary conditions:

$$u(0, \mathbf{x}) = g(\mathbf{x}), \quad (2)$$

$$\mathcal{B}[u] = 0. \quad (3)$$

Here, $t \in [0, T]$, $\mathbf{x} \in \Omega$, Ω is a subset of $\mathbb{R}^D$; $\mathcal{N}[\cdot; \lambda]$ is a linear or nonlinear operator with parameter λ ; $\mathcal{B}$ represents the boundary condition operator; $u(\mathbf{x}, t)$ is the latent (hidden) solution of equation (1).

In the standard PINN, to obtain the unknown solution $u(\mathbf{x}, t)$, one should train the neural network by minimizing the following composite loss function [28]:

$$\mathcal{L}(\theta) = \omega_{\text{PDE}} \mathcal{L}_{\text{PDE}} + \omega_{\text{IC}} \mathcal{L}_{\text{IC}} + \omega_{\text{BC}} \mathcal{L}_{\text{BC}}, \quad (4)$$

where

$$\mathcal{L}_{\text{PDE}}(\theta) = \frac{1}{N_{\text{PDE}}} \sum_{i=1}^{N_{\text{PDE}}} |u_t^i + \mathcal{N}[u_i; \lambda]|^2, \quad (5)$$

$$\mathcal{L}_{\text{IC}}(\theta) = \frac{1}{N_{\text{IC}}} \sum_{i=1}^{N_{\text{IC}}} |u_0(\mathbf{x}_{\text{IC}}^i, 0) - g(\mathbf{x}_{\text{IC}}^i)|^2, \quad (6)$$

$$\mathcal{L}_{\text{BC}}(\theta) = \frac{1}{N_{\text{BC}}} \sum_{i=1}^{N_{\text{BC}}} |\mathcal{B}[u_0(\mathbf{x}_{\text{BC}}^i, t_{\text{BC}}^i)]|^2. \quad (7)$$

Here, $\mathcal{L}_{\text{PDE}}$ (ω_{PDE}), $\mathcal{L}_{\text{IC}}$ (ω_{IC}), and $\mathcal{L}_{\text{BC}}$ (ω_{BC}) are the losses (weights) related to the PDE, initial condition and boundary condition, respectively. The network output u_θ represents the predicted solution by PINN, while θ denotes trainable parameters of the network. The sets $\{(\mathbf{x}_{\text{PDE}}^i, t_{\text{PDE}}^i)\}_{i=1}^{N_{\text{PDE}}}$, $\{(\mathbf{x}_{\text{IC}}^i, 0)\}_{i=1}^{N_{\text{IC}}}$, and $\{(\mathbf{x}_{\text{BC}}^i, t_{\text{BC}}^i)\}_{i=1}^{N_{\text{BC}}}$ correspond to the sampling points for the residual of the equation, initial condition, and boundary condition, respectively.

Usually, the loss function could be minimized via gradient descent methods. To gain a comprehensive understanding of the optimization of PINN, we also illustrate the gradient of the loss function via the $\mathcal{L}_2$ norm, which is written as follows [29]:

$$\mathcal{G} = \sqrt{\sum_i \left(\frac{\partial \mathcal{L}}{\partial \theta_i} \right)^2}. \quad (8)$$

Here, θ_i represents the i th parameter of the neural network.

2.2. The architecture of shared-branch network

In this work, we focus on the ROBER problem, which is characterized by a set of coupled and strongly nonlinear ODEs. Numerical experiments demonstrated that the standard PINN often fails when applied to such problems. To address this problem, we adopt a shared-branch network structure suitable for solving the coupled ROBER problem, recently proposed by Fang *et al* [21].

Typically, a fully connected neural network (FCNN) comprises an input layer, multiple hidden layers, and one output layer. In contrast, the shared-branch network splits the single output layer into multiple parallel ones, aligning its output count with the number of equations to solve, as shown in figure 1. Such an architecture efficiently models the coupling relationships among variables, boosting PINN's performance for complex issues.

3. The ROBER problem

In stiff chemical kinetics, the ROBER problem models autocatalytic reactions and atmospheric processes [4]. It comprises three coupled ODEs tracking species concentrations evolution. In this section, we introduce the governing equations, normalization scheme, and algorithm for our EC-PINN, providing the foundation for subsequent studies.

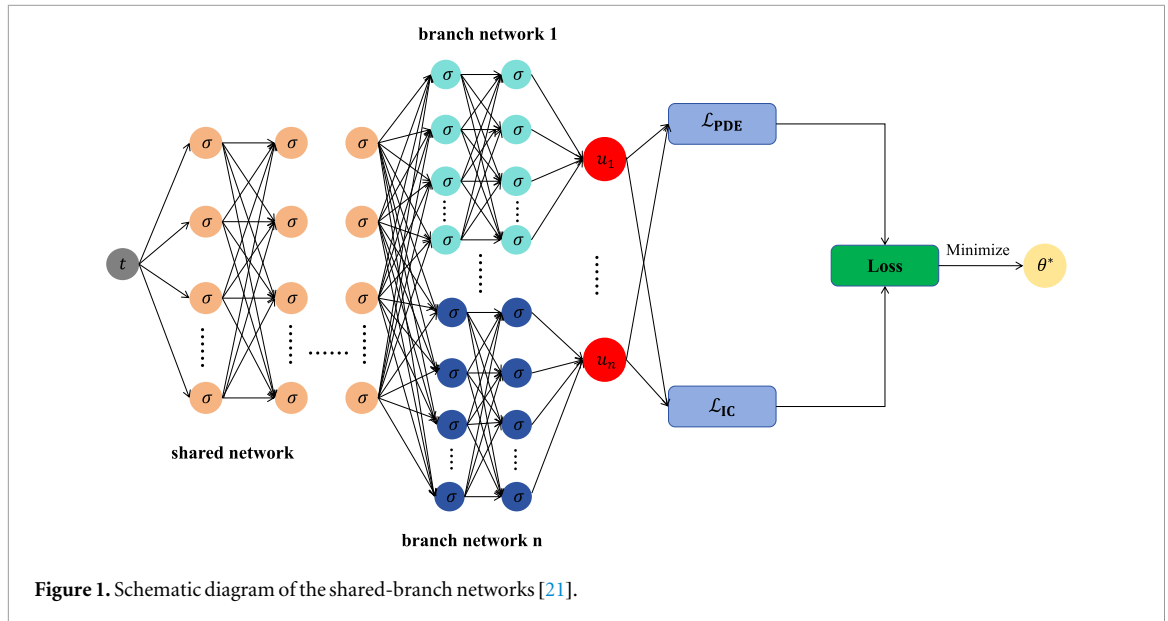
3.1. Normalization scheme of the ROBER problem

For further analysis, we first present the governing equations as follows:

$$\frac{ds_1}{dt} = -k_1 s_1 + k_3 s_2 s_3, \quad (9)$$

$$\frac{ds_2}{dt} = k_1 s_1 - k_2 s_2^2 - k_3 s_2 s_3, \quad (10)$$

$$\frac{ds_3}{dt} = k_2 s_2^2, \quad (11)$$



where $t \in [0, T]$, s_1 , s_2 , and s_3 denote chemical species concentrations, with k_1 , k_2 , and k_3 as reaction rate constants. The initial conditions adopted in this work are listed as follows:

$$s_1(0) = 1, \quad s_2(0) = 0, \quad s_3(0) = 0. \quad (12)$$

It should be noted that, all quantities (e.g., t , β , k_i , and s_i , where $i = 1, 2, 3$) are dimensionless. More interesting, when we directly sum equations (9), (10), and (11), the following conservation law is obtained:

$$\frac{dE}{dt} = \frac{d}{dt}(s_1 + s_2 + s_3) = 0, \quad (13)$$

which was recently proposed by Fang *et al* [21]. Combined with the initial conditions, we arrive at:

$$E = s_1(t) + s_2(t) + s_3(t) = s_1(0) + s_2(0) + s_3(0) = 1. \quad (14)$$

In fact, the prior study has demonstrated that integrating this conservation law into the loss function significantly enhances PINN's performance on such a task [21].

Usually, in the ROBER problem, reaction rate constants differ by several orders of magnitude, which leads to stiffness of the system. Similar to the previous work [19], we set these parameters as: $k_1 = 4 \times 10^{-2}$, $k_2 = 3 \times 10^7$, and $k_3 = 1 \times 10^4$. To mitigate the stiffness of the ROBER problem, an appropriate normalization scheme is needed. Following [21], we define: $\tilde{s}_2 = \beta^{-1}s_2$, $\tilde{k}_2 = \beta^2k_2$, and $\tilde{k}_3 = \beta k_3$, where $\beta = 10^{-4}$.

Then, the governing equations (equations (9) to (12)) can be rewritten as:

$$\frac{ds_1}{dt} = -k_1s_1 + \tilde{k}_3\tilde{s}_2s_3, \quad (15)$$

$$\beta \frac{d\tilde{s}_2}{dt} = k_1s_1 - \tilde{k}_2\tilde{s}_2^2 - \tilde{k}_3\tilde{s}_2s_3, \quad (16)$$

$$\frac{ds_3}{dt} = \tilde{k}_2\tilde{s}_2^2. \quad (17)$$

Simultaneously, the corresponding initial conditions are:

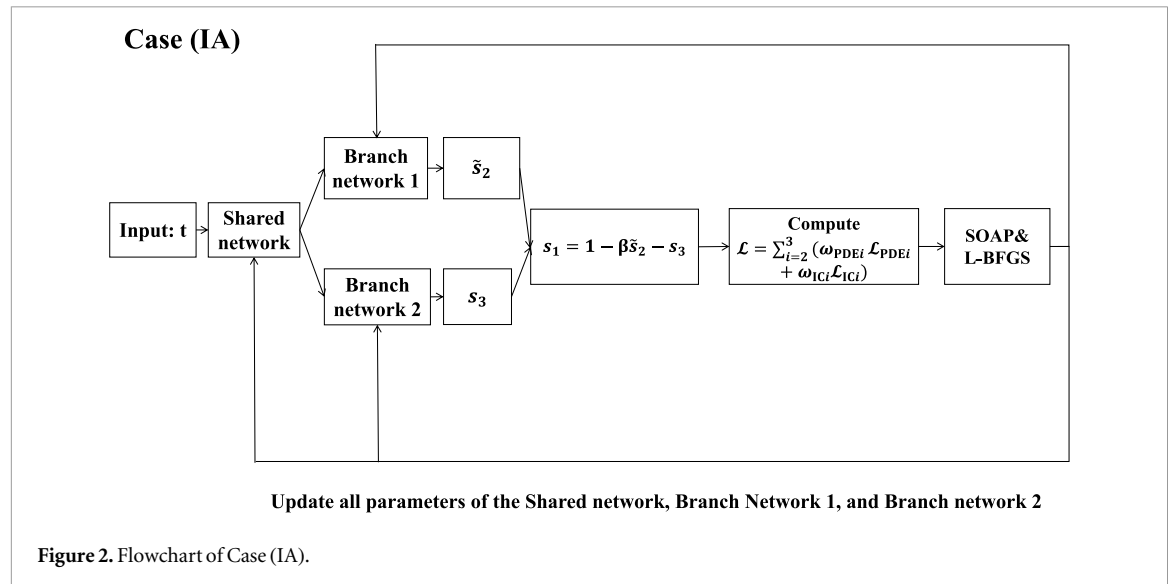
$$s_1(0) = 1, \quad \tilde{s}_2(0) = 0, \quad s_3(0) = 0. \quad (18)$$

Now, the conservation law should be rewritten as:

$$E = s_1(t) + \beta\tilde{s}_2(t) + s_3(t) = 1. \quad (19)$$

Finally, to evaluate the accuracy of the predicted solution, we adopt the relative error formula based on the L_2 norm, which can be expressed as follows [30]:

$$\text{Relative error} = \frac{\sqrt{\sum_{i=1}^{N_t} |u(t_i) - u_\theta(t_i)|^2}}{\sqrt{\sum_{i=1}^{N_t} |u(t_i)|^2}}. \quad (20)$$



where u and u_θ are the ground truth and predicted solutions of the governing equations, N_t is the number of sampling points.

3.2. A short review of CC-PINN

Although our normalization partially alleviates the stiffness of the ROBER problem, the standard PINN may still encounter optimization challenges when applied to such a multi-scale, strongly nonlinear stiff system. On the other hand, in chemical reactions, the concentration of substances is conserved, as mentioned in section 3.1. To improve the performance of PINN, Fang *et al* proposed the CC-PINN strategy, explicitly adding the conservation law (equation (19)) as an additional term to the loss function [21]. For clarity, we write the proposed loss function as follows:

$$\mathcal{L}(\theta) = \sum_{i=1}^3 \omega_{\text{PDE}i} \mathcal{L}_{\text{PDE}i} + \sum_{i=1}^3 \omega_{\text{IC}i} \mathcal{L}_{\text{IC}i} + \omega_{\text{CON}} \mathcal{L}_{\text{CON}}, \quad (21)$$

where

$$\mathcal{L}_{\text{PDE}1} = \frac{1}{N_{\text{PDE}1}} \sum_{i=1}^{N_{\text{PDE}1}} \left| \frac{ds_1(t)}{dt} + k_1 s_1(t) - \tilde{k}_3 \tilde{s}_2(t) s_3(t) \right|^2, \quad (22)$$

$$\mathcal{L}_{\text{PDE}2} = \frac{1}{N_{\text{PDE}2}} \sum_{i=1}^{N_{\text{PDE}2}} \left| \beta \frac{d\tilde{s}_2(t)}{dt} - k_1 s_1(t) + \tilde{k}_2 \tilde{s}_2^2(t) + \tilde{k}_3 \tilde{s}_2(t) s_3(t) \right|^2, \quad (23)$$

$$\mathcal{L}_{\text{PDE}3} = \frac{1}{N_{\text{PDE}3}} \sum_{i=1}^{N_{\text{PDE}3}} \left| \frac{ds_3(t)}{dt} - \tilde{k}_2 \tilde{s}_2^2(t) \right|^2, \quad (24)$$

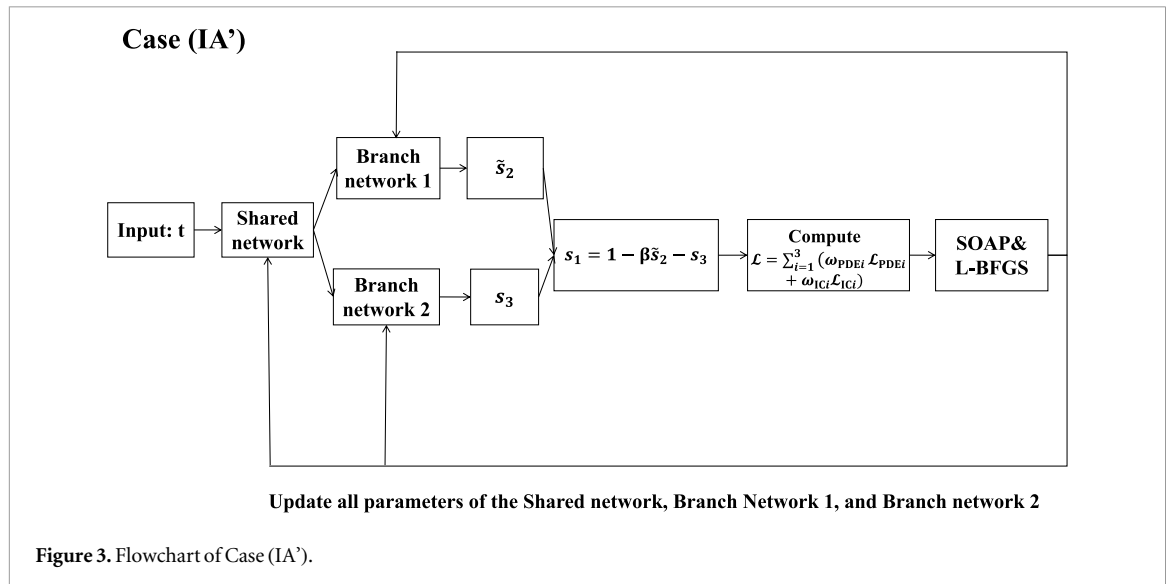
$$\mathcal{L}_{\text{IC}1} = \frac{1}{N_{\text{IC}1}} \sum_{i=1}^{N_{\text{IC}1}} |s_1(0) - 1|^2, \quad (25)$$

$$\mathcal{L}_{\text{IC}2} = \frac{1}{N_{\text{IC}2}} \sum_{i=1}^{N_{\text{IC}2}} |\tilde{s}_2(0)|^2, \quad (26)$$

$$\mathcal{L}_{\text{IC}3} = \frac{1}{N_{\text{IC}3}} \sum_{i=1}^{N_{\text{IC}3}} |s_3(0)|^2, \quad (27)$$

$$\mathcal{L}_{\text{CON}} = \frac{1}{N_{\text{CON}}} \sum_{i=1}^{N_{\text{CON}}} |s_1(t) + \beta \tilde{s}_2(t) + s_3(t) - 1|^2. \quad (28)$$

Here, the total loss combines the conservation constraint term ($\mathcal{L}_{\text{CON}}$), residual terms from equations (15)–(17) ($\mathcal{L}_{\text{PDE}1}$, $\mathcal{L}_{\text{PDE}2}$, and $\mathcal{L}_{\text{PDE}3}$), and initial condition terms for s_1 , $\tilde{s}_2$, and s_3 ($\mathcal{L}_{\text{IC}1}$, $\mathcal{L}_{\text{IC}2}$, and $\mathcal{L}_{\text{IC}3}$). The weights and the number of sampling points in these seven terms are $\omega_{\text{PDE}1}$, $\omega_{\text{PDE}2}$, $\omega_{\text{PDE}3}$, $\omega_{\text{IC}1}$, $\omega_{\text{IC}2}$, $\omega_{\text{IC}3}$, ω_{CON} , and $N_{\text{PDE}1}$, $N_{\text{PDE}2}$, $N_{\text{PDE}3}$, $N_{\text{IC}1}$, $N_{\text{IC}2}$, $N_{\text{IC}3}$, N_{CON} , respectively.



3.3. The algorithm of EC-PINN

To solve the ROBER problem, Fang *et al* proposed a CC-PINN strategy [21]. Numerical experiments demonstrated this method enhances both the stability and accuracy of PINN solutions, although predictions of conserved quantities still exhibit some deviation. It should be noted that, such a method uses a ‘soft’ optimization-based constraint, which fails to completely eliminate deviations in conserved quantities. To improve the accuracy and stability of PINN solutions, we propose an Exact-Constraint strategy, where conservation law is enforced as exact constraint in the loss function. Now, we present the implementation of exact constraint within PINN.

3.3.1. The loss function of pure exact constraint strategy

Based on the conservation law, any quantity (such as s_1) can be expressed in terms of the other two variables (herein, $\tilde{s}_2$ and s_3) from equation (19). This observation enables us to avoid calculating indirect variables (such as s_1) during the training procedure. Instead, we derive their values from direct variables and the conservation law (herein, $s_1 = 1 - \beta \tilde{s}_2 - s_3$) in the output stage. For simplicity, this scheme is referred to as Case (IA). Similarly, the other two exact conservation approaches that avoid directly computing either $\tilde{s}_2$ or s_3 are labeled as Case (IIA) or Case (IIIA), which can be found in Appendix A.

In Case (IA), the total loss function $\mathcal{L}(\theta)$ comprises the following components:

$$\mathcal{L}(\theta) = \sum_{i=2}^3 \omega_{PDEi} \mathcal{L}_{PDEi} + \sum_{i=2}^3 \omega_{ICi} \mathcal{L}_{ICi}. \quad (29)$$

The flowchart of Case (figure 2) (IA) is as follows:

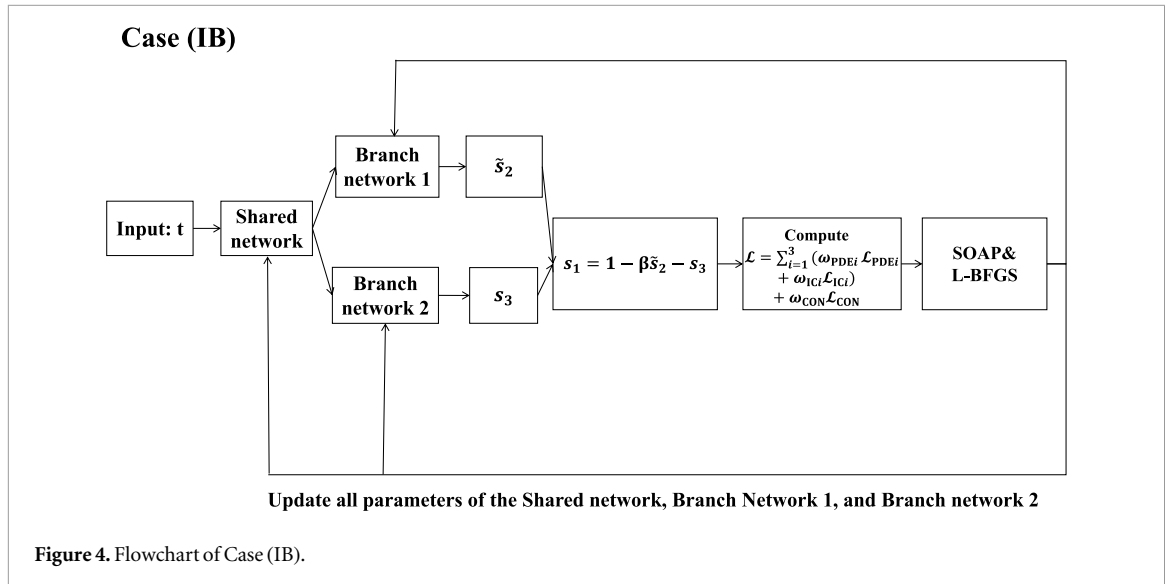
Now we give some remarks on such an approach. In Case (IA), the loss function is straightforward and easy to implement. Reducing the number of loss terms makes the optimization simpler and lowers computational cost. However, indirect variables are not explicitly constrained, which may make the training of PINN more difficult.

To further enhance the performance of our EC-PINN, we propose another formulation of the loss function termed Case (IA'). Different from Case (IA), the new loss function in Case (IA') preserves all terms associated with the indirect variable (herein, s_1) during the training procedure. It further incorporates the conservation law with direct variable predictions ($\tilde{s}_2$ and s_3) to derive the indirect variable (s_1) during the output stage. Similarly, two scenarios, i.e. Case (IIA') and Case (IIIA'), will be established in Appendix B. Specifically, the total loss function $\mathcal{L}(\theta)$ is expressed as follows:

$$\mathcal{L}(\theta) = \sum_{i=1}^3 \omega_{PDEi} \mathcal{L}_{PDEi} + \sum_{i=1}^3 \omega_{ICi} \mathcal{L}_{ICi}. \quad (30)$$

The flowchart corresponding to Case (IA') (figure 3) is presented as follows:

Different from Case (IA), this method integrates $\mathcal{L}_{PDE1}$ and $\mathcal{L}_{IC1}$ into the loss function. During the training procedure, such terms explicitly provide constraints on the indirect variable s_1 . During the output stage, we calculate s_1 via the conservation law ($s_1 = 1 - \beta \tilde{s}_2 - s_3$). Generally, this method improves the performance of PINN to some extent, which will be presented in section 4.



3.3.2. The loss function of hybrid constraints strategy

In order to further improve the EC-PINN's performance, we propose its second scenario Case (IB). Similar to Case (IA') in section 3.3.1, this approach also adopts s_1 as an indirect variable. Meanwhile, the soft constraint on the conservation law (denoted by $\mathcal{L}_{\text{CON}}$; see equation (28)) is applied at all training iteration steps. We call this combination the hybrid constraints strategy. Specifically, the total loss function $\mathcal{L}(\theta)$ is expressed as follows:

$$\mathcal{L}(\theta) = \sum_{i=1}^3 \omega_{\text{PDE}i} \mathcal{L}_{\text{PDE}i} + \sum_{i=1}^3 \omega_{\text{IC}i} \mathcal{L}_{\text{IC}i} + \omega_{\text{CON}} \mathcal{L}_{\text{CON}}. \quad (31)$$

The flowchart for Case (IB) (figure 4) is given below:

Some remarks on Case (IB) are provided below. Different from Case (IA'), we implement the hybrid constraints strategy in Case (IB). This method adopts the exact constraint from Case (IA'). And it simultaneously incorporates the conservation law term (see equation (28)) into the loss function, ensuring that the conservation law is applied at each training step. Generally, this strategy improves the performance of PINN in terms of physical consistency and accuracy. The specific experimental results will be presented in section 4. It should be noted that, the expressions of loss functions for Case (IIB) and Case (IIIB) are similar to that of Case (IB), which can be found in Appendix C for details.

For clarity, we illustrate the EC-PINN algorithm and settings using the pure exact constraint strategy and the hybrid constraints strategy. The algorithm is summarized as follows:

Algorithm 1. EC-PINN Algorithm

- 1: **Normalization scheme:** Transform the ROBER problem appropriately.
- 2: **Input:**
- 3: The architecture of shared-branch network: 7 layers in the shared network and 5 layers in the branch network;
- 4: Time domain $[0, T]$;
- 5: **Construct the loss function:**
- 6: The pure exact constraint strategy:
- 7: Case (IA) : Apply constraints to the direct variables ($\tilde{s}_2$ and s_3) while excluding terms of indirect variables (s_1) (equation (29));
- 8: Case (IA') : Retain all terms of direct and indirect variables (equation (30));
- 9: The hybrid constraints strategy (Case (IB)): Built on Case (IA') with the soft constraint applied as specified in equation (31);
- 10: **for** $k = 1: n$ **do**
- 11: Use SOAP to minimize (equation (29), (30) or (31));
- 12: Switch the optimizer based on iteration steps;
- 13: Calculate the indirect variable (s_1) combined with the direct variables ($\tilde{s}_2$ and s_3) and conservation law;
- 14: **end for**

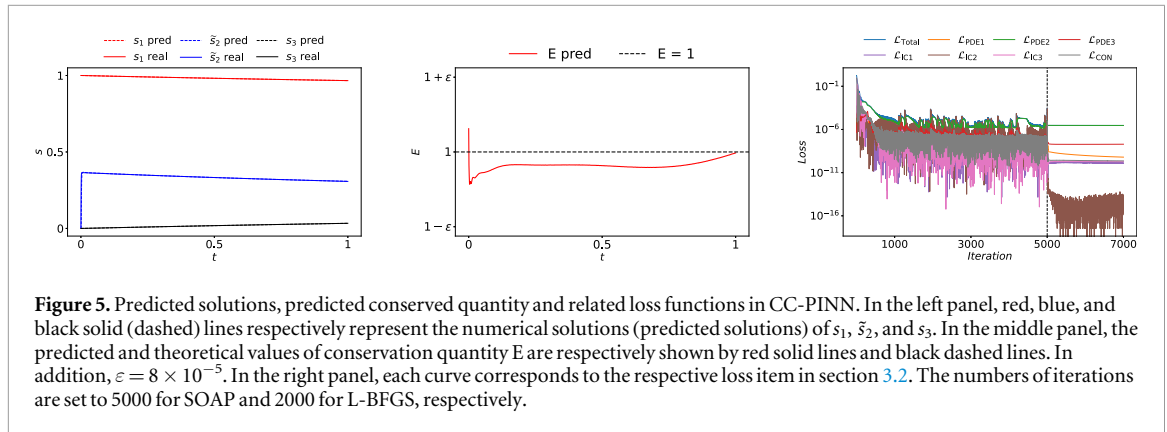


Figure 5. Predicted solutions, predicted conserved quantity and related loss functions in CC-PINN. In the left panel, red, blue, and black solid (dashed) lines respectively represent the numerical solutions (predicted solutions) of s_1 , s_2 , and s_3 . In the middle panel, the predicted and theoretical values of conservation quantity E are respectively shown by red solid lines and black dashed lines. In addition, $\epsilon = 8 \times 10^{-5}$. In the right panel, each curve corresponds to the respective loss item in section 3.2. The numbers of iterations are set to 5000 for SOAP and 2000 for L-BFGS, respectively.

Here, the indirect variable is chosen as s_1 , which corresponds to Case (IA), Case (IA') and Case (IB). However, the aforementioned algorithm can be easily extended to cases where the indirect variables are s_2 and s_3 .

In our numerical experiments, we set $T = 1$. Computations utilize the deep learning framework PyTorch (with float32 precision). Experiments are implemented on a shared-branch network architecture powered by an NVIDIA GeForce RTX 4070 Ti SUPER GPU. There are 6 hidden layers in the shared network. Meanwhile, the number of branch network corresponds to that of PDEs to be solved. In each of these branch networks, there are 4 hidden layers. Each layer contains 100 neurons. The activation function is tanh. And the number of sampling points is 1000. During training, we employ a combined SOAP and L-BFGS optimization strategy [31–33]. Learning rates are set to 1×10^{-3} for SOAP and 1 for L-BFGS, respectively. Accordingly, the gradients, weights, and biases of the FCNN are automatically computed and updated by the employed optimizers, while the calculation details of these parameters can be found in [31, 34].

In addition, adaptive weight strategies have been extensively explored in PINN and shown to significantly affect PINN's performance. In this work, we focus on the effectiveness of the exact constraint. Consequently, all loss term weights are specified as 1.

4. Numerical experiments

In this section, we solve the ROBER problem using our EC-PINN strategy. Specifically, we implement the pure exact conservation and the hybrid constraints strategy to enhance the performance of PINN. Numerical experiments demonstrate that the hybrid constraints approach achieves higher solution accuracy than the pure method. In addition, we calculate the correlation coefficients among the loss function, its gradient and the relative error. Results indicate that solving PDEs through loss function minimization in PINN is reasonable.

4.1. The pure exact constraint strategy

Recently, Fang *et al* used the CC-PINN method for the ROBER problem [21]. In their work, the relative errors of s_1 , s_2 , and s_3 are 6.36×10^{-5} , 1.59×10^{-2} , and 4.17×10^{-3} , respectively. It should be noted that, in their work, a combination of Adam and L-BFGS optimizers was adopted [35]. Instead, we optimize the neural network via the combination of SOAP and L-BFGS, which can improve the performance of PINN to a certain degree. As shown in figure 5, the total loss converges to a relatively small value when the number of iteration steps is appropriate. When the loss function converges, the predicted results for s_1 , s_2 , and s_3 agree well with numerical solutions, as illustrated in figure 5. In this situation, the relative errors of s_1 , s_2 , and s_3 are 1.10×10^{-5} , 2.85×10^{-2} , and 1.24×10^{-3} , respectively. Hereafter, our EC-PINN will be implemented based on the combination of SOAP and L-BFGS.

During the training procedure of PINN, it is believed that the convergence of the loss function indicates that we have obtained the predicted solution of the unsolved PDEs with certain initial and boundary conditions [8]. In our EC-PINN, SOAP is used to minimize the loss function at first. After 5000 training steps, the optimizer will be switched to L-BFGS. At this stage, the loss function drops sharply and converges quickly. Although we cannot provide a comprehensive understanding of this phenomenon, based on the results presented in these figures, we tend to believe that this fact may be due to the local convergence of the L-BFGS algorithm. However, a more detailed study of this interesting phenomenon is beyond the scope of this work, which will be presented in the future.

While the CC-PINN has shown promising performance, the 'soft' constraint strategy always leads to the violation of the conservation law (see also the middle panel of figure 5), which may limit the accuracy and efficiency of PINN. To address this issue, we propose a new strategy named EC-PINN. As mentioned in

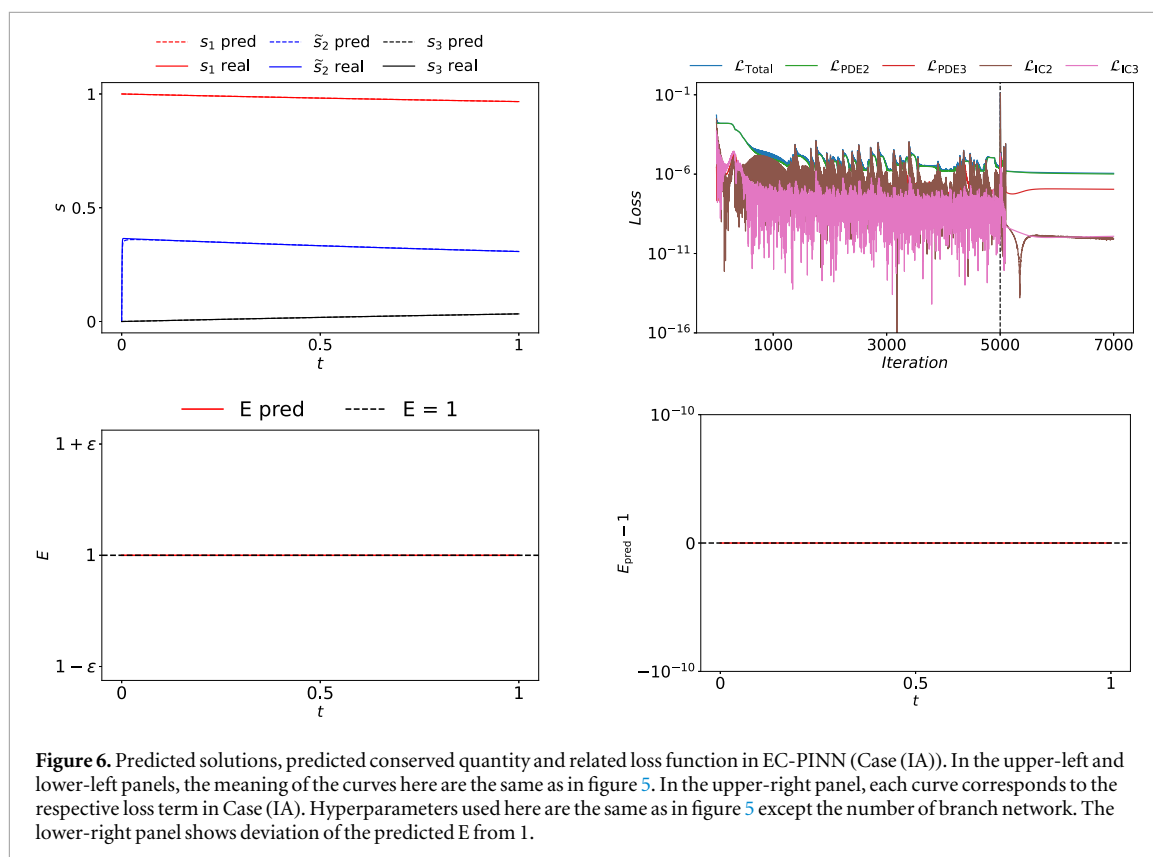


Figure 6. Predicted solutions, predicted conserved quantity and related loss function in EC-PINN (Case (IA)). In the upper-left and lower-left panels, the meaning of the curves here are the same as in figure 5. In the upper-right panel, each curve corresponds to the respective loss term in Case (IA). Hyperparameters used here are the same as in figure 5 except the number of branch network. The lower-right panel shows deviation of the predicted E from 1.

section 3.3, there are two distinct constraint strategies in EC-PINN. In this subsection, we examine the performance of the first approach.

In the first approach, the PDE and initial losses of the indirect variable (such as s_1) are not included in the total loss function. Instead, we calculate the indirect variable via the conservation law (equation (19)) in the output stage. Notably, s_1 exhibits lower computational complexity than $\tilde{s}_2$, and s_3 (see also in section 3.3.1 and the left panel of figure 5). Therefore, in this subsection, we choose s_1 as the indirect variable, which is named Case (IA). The numerical experimental results of Case (IA) are presented in figure 6.

Similar to figure 5, the predicted results show excellent agreement with numerical solutions. Furthermore, relative errors of s_1 , $\tilde{s}_2$ and s_3 are 8.81×10^{-5} , 8.45×10^{-3} , and 4.29×10^{-3} , respectively. Notably, compared to the case in figure 5, the relative error of $\tilde{s}_2$ is improved by 70.35%, however, the accuracy of s_1 and s_3 decreases. Even though, our EC-PINN is meaningful since the relative error of the stiff variable $\tilde{s}_2$ is significantly improved. On the other hand, in both CC-PINN and EC-PINN, the accuracy of s_1 and s_3 is acceptable. Therefore, we tend to believe that the general performance of our EC-PINN is superior to the previous work [21].

As expected, in the lower panel of figure 6, one can find that the deviation of predicted E from 1 is identically zero, which indicates that E is rigorously conserved. This is different from cases in CC-PINN (see the middle panel of figure 5). Additionally, from the upper-right panel of figure 6, we can clearly see the $\mathcal{L}_{PDE2}$ is dominant during the entire training procedure. This is not surprising, since $\tilde{s}_2$ is the stiff variable in the ROBER problem. In fact, this observation can also be found in figure 7. As mentioned above, by implementing an exact constraint of the conservation law, we significantly enhance the stiff variable's accuracy. In fact, since the conservation law is exactly implemented, we may expect that this strategy can improve the capability of long time integration. Therefore, we may provide a reliable approach for solving stiff chemical kinetics.

On the other hand, compared with CC-PINN, the accuracy of s_1 and s_3 unexpectedly drops. This might arise from the computational approach we employed. Here, we do not constrain s_1 during each training step. Instead, we calculate the indirect variable in the output stage based on the conservation law. However, as $\tilde{s}_2$ is a stiff variable, its computational accuracy stays low even after optimization is completed. Therefore, it may be unreasonable to derive s_1 from $\tilde{s}_2$, and s_3 only at the output stage without imposing constraints on s_1 during each training step. Meanwhile, s_1 actually appears in the equation for $\tilde{s}_2$. Consequently, the inaccuracy of s_1 will further increase the overall training difficulty.

To enhance the performance of our EC-PINN, during each training step, we retain the equations for s_1 , $\tilde{s}_2$, and s_3 simultaneously. At the output stage, however, we compute s_1 using $\tilde{s}_2$, s_3 and the conservation law (equation (19)). To clarify, we call it Case (IA'). Similarly, two scenarios, i.e. Case (IIA') and Case (IIIA'), can be

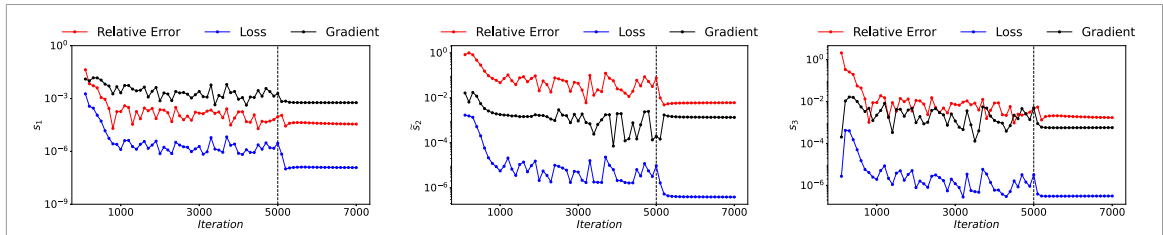


Figure 8. Loss function, its gradient and relative error of s_1 , s_2 , and s_3 with iteration steps (the first method). In the three panels, red, blue, and black curves respectively represent the relative error, loss, and its gradient. Hyperparameters used here are the same as in figure 5.

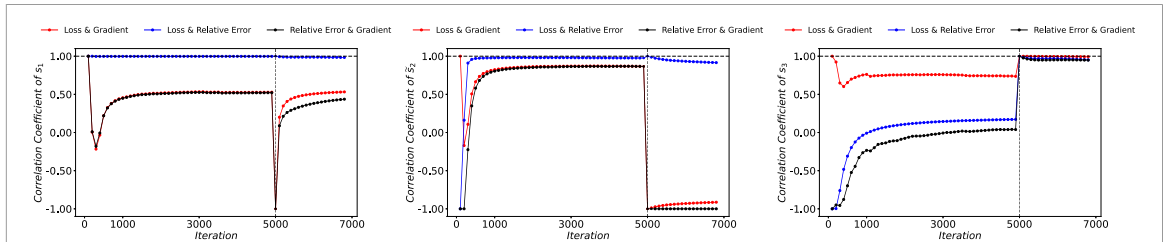


Figure 9. Correlation coefficients among PDE loss, its gradient and relative error of s_1 , s_2 , and s_3 with iteration steps. The red, blue, and black curves represent the correlation coefficients between loss and gradient, loss and relative error, relative error and gradient, respectively. Hyperparameters used here are the same as in figure 5.

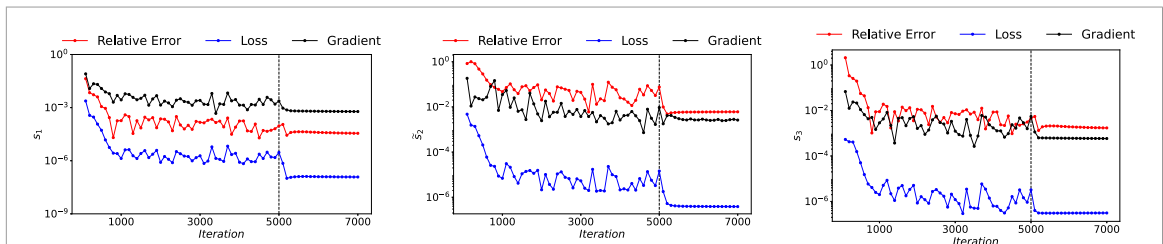


Figure 10. Loss function, its gradient and relative error of s_1 , s_2 , and s_3 with iteration steps (the second method). The meaning of the curve colors here is the same as in figure 8. Hyperparameters used here are the same as in figure 5.

meanwhile, gradients and relative errors of s_1 , s_2 , and s_3 also approach to steady states. At this time, the optimization of PINN is completed. Therefore, evaluating PINN's performance via the reduction and convergence of loss function is reasonable. Nevertheless, the loss function, its gradient, and relative error do not exhibit a one-to-one relationship.

To clarify, we illustrate the correlation coefficients of s_1 , s_2 , and s_3 . Clearly, in the SOAP stage, correlation coefficients of loss and relative error for s_1 and s_2 near 1. However, the correlation coefficient of s_3 's relative error and PDE loss is about 0.19. Generally speaking, relative errors show a close relationship with PDE losses. Furthermore, the relationship between relative errors and PDE losses is stronger than that between relative errors and the gradients of the PDE loss. Surprisingly, the correlation between PDE loss and its gradient is not as strong as one might expect.

Now let's discuss the relationship between the total loss, its gradient, and the relative error of s_1 , s_2 , and s_3 . Similar to figure 8, total loss of s_1 , s_2 , and s_3 decreases with iteration steps. Similarly, PDE losses converge to sufficiently small values once the optimizer switches from SOAP to L-BFGS; simultaneously, gradient and relative error of s_1 , s_2 , and s_3 also approach to steady states, which leads to the completion of the optimization.

Similar to figure 9, in figures 10 and 11, the correlation coefficient between the relative error and the total loss of s_1 approaches nearly 1. Meanwhile, the remaining two correlation coefficients increase sharply from about 0.5 to nearly 1. This further confirms the relationship between the relative error and the total loss. Compared with figure 9, all three correlation coefficients of s_2 in figure 11 decrease. Conversely, all three correlation coefficients of s_3 exhibit an upward trend. Generally speaking, these findings indicate a high correlation between the relative error and the total loss function of s_1 , s_2 , and s_3 .

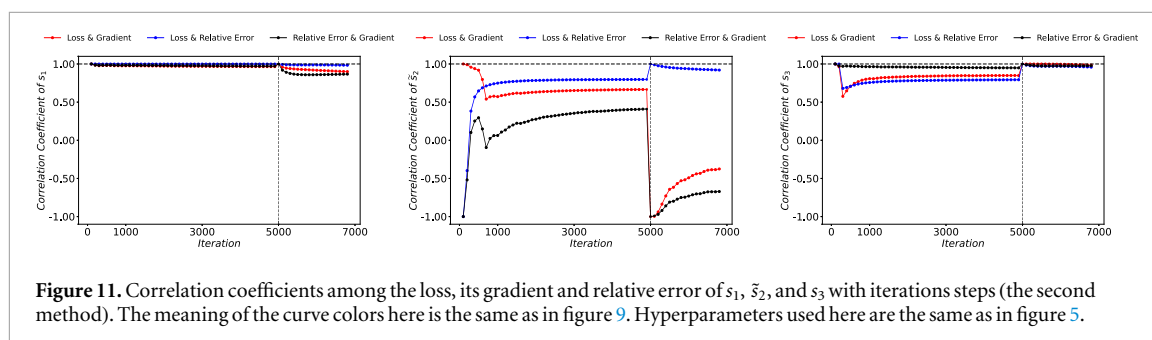


Figure 11. Correlation coefficients among the loss, its gradient and relative error of s_1 , s_2 , and s_3 with iterations steps (the second method). The meaning of the curve colors here is the same as in figure 9. Hyperparameters used here are the same as in figure 5.

Table 2. Relative errors and calculation time under different forms of EC-PINN.

Case form	Relative error of s_1	Relative error of s_2	Relative error of s_3	Time (min)
Case (IA')	2.23×10^{-5}	2.11×10^{-2}	1.08×10^{-3}	3.82
Case (IB)	3.51×10^{-5}	6.09×10^{-3}	1.70×10^{-3}	3.37
Case (IIA')	2.69×10^{-3}	9.43×10^{-1}	9.97×10^{-1}	2.85
Case (IIB)	2.80×10^{-3}	9.72×10^{-1}	7.62×10^{-1}	2.94
Case (IIIA')	NaN	NaN	NaN	—
Case (IIIB)	7.78×10^{-6}	1.79×10^{-2}	3.51×10^{-4}	3.06

As mentioned earlier, we adopt the combination of SOAP and L-BFGS. Specifically, the SOAP optimizer decreases the loss function with sharp oscillations; when switching to L-BFGS, the loss function quickly stabilizes at a small value (see the right panels of figure 5 and the upper-right panel of figure 6). Usually, optimizers such as SOAP and Adam can only yield approximate solutions for PINN [8, 30]. To obtain more accurate solutions, we should combine multiple optimizers appropriately. Therefore, a critical challenge here is determining the switch mechanism.

As shown in figures 9 and 11, relative errors and loss functions show strong correlations. As noted earlier, Adam or SOAP only yields approximate solutions for PINN. To achieve high-accuracy results, we should switch the optimizer to L-BFGS when the loss is sufficiently small. In previous studies, the selection of the switching step relies on empirical rules, which lack a theoretical basis [18]. In the future, we will focus on developing a theoretically guided formula for switching mechanism, aiming to improving PINN's interpretability.

Finally, we also set s_2 (Case (IIA') and Case (IIB)) and s_3 (Case (IIIA') and Case (IIIB)) as the indirect variables. To provide a comprehensive understanding on our EC-PINN, we summarize the results in table 2. As shown in table 2, performance of Case (IB) is significantly superior to that of other cases. On the one hand, compared with s_2 and s_3 , s_1 has the lowest computational complexity (see figure 5). Consequently, it is more suitable than s_2 and s_3 to be used as an indirect variable. On the other hand, it is meaningful to incorporate soft conservation constraint during iteration steps of PINN. In fact, these constraints can significantly improve PINN's performance.

5. Summary and discussion

5.1. Summary

In this work, we propose a new framework, i.e., EC-PINN, to address the ROBER problem in stiff chemical kinetics. Specifically, the EC-PINN integrates the conservation law as an exact constraint, enforcing rigorous conservation during the training procedure. In this framework, we implement both the pure exact and the hybrid constraint strategies to enhance the performance of PINN. In the pure exact constraint approach, we further present two distinct formulations. The first formulation simplifies implementation by eliminating the governing equations and initial conditions for indirect variables, applying constraints only to the direct ones. In contrast, the second formulation retains all indirect variable terms during the training procedure. It should be noted that, both formulations integrate the conservation law with direct variable predictions to compute indirect variables at the output stage. On this basis, the hybrid constraints strategy applies the soft constraint on the conservation law at all training iteration steps. Numerical experiments demonstrate that the hybrid constraints approach achieves higher solution accuracy than the pure method. Consequently, it is meaningful to incorporate soft conservation constraint during iteration steps of PINN. In fact, these constraints can

Table 3. Relative errors and calculation time for EC-PINN with different number of shared network layers (Case (IA)).

Number of layers in shared network	Number of layers in branch network	Time (min)	Relative error of s_1	Relative error of $\tilde{s}_2$	Relative error of s_3
12	0	0.70	1.67×10^{-3}	1.76×10^{-1}	8.14×10^{-2}
11	1	2.91	8.96×10^{-5}	9.51×10^{-3}	4.36×10^{-3}
10	2	3.39	4.84×10^{-5}	4.92×10^{-3}	2.37×10^{-3}
9	3	2.73	8.44×10^{-5}	4.96×10^{-2}	4.11×10^{-3}
8	4	2.64	2.35×10^{-5}	1.76×10^{-2}	1.14×10^{-3}
7	5	3.72	8.81×10^{-5}	8.45×10^{-3}	4.29×10^{-3}
6	6	3.12	1.60×10^{-5}	6.16×10^{-3}	7.65×10^{-4}
5	7	2.97	5.06×10^{-6}	7.84×10^{-3}	1.99×10^{-4}
4	8	3.22	2.53×10^{-5}	2.18×10^{-2}	1.22×10^{-3}
3	9	3.21	2.93×10^{-5}	1.19×10^{-2}	1.42×10^{-3}
2	10	3.33	1.14×10^{-4}	8.25×10^{-3}	5.54×10^{-3}
1	11	3.57	3.86×10^{-5}	1.60×10^{-2}	1.87×10^{-3}

significantly improve PINN's performance. Furthermore, compared with the prior CC-PINN method [21], EC-PINN not only improves global performance but also strictly preserves the conservation law.

To improve the interpretability of PINNs, we calculate correlation coefficients among the loss function, its gradient and the relative error using two methods. Specifically, one approach only considers the dependencies between the relative error, the PDE loss function and its gradient, while the other focuses on the dependencies between the relative error, the total loss function (including the PDE loss and the initial loss) and its gradient. Results indicate that a high correlation between the relative error and the total loss function for s_1 , $\tilde{s}_2$, and s_3 , validating that solving PDEs through loss minimization in PINN is reasonable.

In summary, we propose the EC-PINN for solving the stiff chemical kinetics ROBER problem. This method designs two pure exact constraint strategies and further proposes a hybrid constraint strategy. The results demonstrate that the latter's solution accuracy is significantly better than that of the former. On the other hand, the correlation analysis verifies the rationality of PINN in solving PDEs through loss minimization. Compared with CC-PINN, this method not only improves overall performance but also strictly maintains the conservation law. It provides a novel approach for solving stiff systems, particularly in the long time integration tasks.

5.2. Discussion

Recently, solving stiff chemical kinetics via PINN has attracted widespread attention. To overcome the stiffness of the original system, various variants of PINN such as Stiff-PINN [19], MPINN [20], and CC-PINN [21] were proposed. In Stiff-PINN [19], Quasi-Steady-State-Assumptions (QSSA) is employed to reduce the stiffness. It should be noted that, under QSSA, the stiff variable ($\tilde{s}_2$ can be expressed in terms of other variables (s_1 and s_3). Similarly, the indirect variable in this study (e.g., s_1 in Case (IA)) can also be represented by the direct variables ($\tilde{s}_2$ and s_3). From this perspective, the two approaches are quite similar. However, our EC-PINN retains all terms in the governing equations. Meanwhile, MPINN introduces branch networks to group and train chemical species with different time scales, thereby mitigating the stiffness of the original issue. Unlike MPINN [20], we utilize branch networks to address coupling issues among the ODEs. We mitigate the stiffness of the governing equations by adopting a normalization scheme [18, 22]. On the other hand, to improve the performance of PINN, Fang *et al* introduced CC-PINN [21]. Although the accuracy of PINN has been significantly improved, the 'soft' constraint strategy always leads to the violation of the conservation law, which may limit the performance of PINN in long time integration tasks. In this work, we propose an exact constraint strategy. By introducing the exact constraint approach, we strictly enforce the conservation law during the entire training procedure. Although our work may seem similar to the studies in [19–21], as discussed earlier, our method differs fundamentally from these previous works. Meanwhile, comparisons between CC-PINN and our EC-PINN have been presented in the section 4.1. However, further comparisons with other methods are not included here, as we focus on the impact of exact constraint on PINN's performance. We believe that this approach could be combined with other methods to further improve the performance of PINN, which we will consider in the future work.

Additionally, we discuss uncertainties in model parameters like the number of shared network layers, using Case (IA) as an example (results in table 3 of Appendix D). From the table 3, different layer combinations significantly affect calculation accuracy and costs, yet there is no guiding principle for selecting the layer number. Consequently, this study also has certain limitations. On one hand, systematic parameter tuning and sensitivity analysis were not performed. On the other hand, this study adopts the FCNN architecture. In the future, we will carry out further analysis of these parameters as well as sensitivity analysis. Meanwhile, more advanced

frameworks such as Long Short-Term Memory (LSTM) network [36], Transformer [37, 38], and fuzzy based models [39–41] will be introduced in conjunction with our current work, which have the potential to further enhance PINN's performance. In addition, our methodology can be extended to model physical processes in multidisciplinary domains, such as combustion science, atmospheric chemistry, and environmental engineering [25, 42, 43]. We tend to believe that PINN integrated with exact constraint may hold potential for offering new insights into simulation studies in related fields.

Acknowledgments

This work was supported by the National Key R&D Program of China under ContractNo. 2024YFE03240300 and the Project of Anhui International Joint Research Center for Metallurgical Process and System Science, Anhui University of Technology No.2024004.

Data availability statement

The data that support the findings of this study are openly available at the following URL/DOI: <https://github.com/huangwenlongplasma/EC-PINN>.

Appendix A. The loss function of Case (IIA) and Case (IIIA)

In section 3.3.1, we introduced a pure exact constraint form of Case (IA) (indirectly computing s_1). Here, we propose an alternative scenario, i.e., Case (IIA), which directly predicts variables (s_1, s_3) and employs the conservation law ($\beta\tilde{s}_2(t) = 1 - s_1(t) - s_3(t)$) to derive the indirect variable $\tilde{s}_2$.

Specifically, Case (IIA) excludes the governing equation and initial condition of the indirect variable $\tilde{s}_2$, and applies constraints solely to the directly predicted variables (s_1, s_3).

In Case (IIA), the total loss function $\mathcal{L}(\theta)$ comprises the following components:

$$\mathcal{L}(\theta) = \omega_{\text{PDE1}} \mathcal{L}_{\text{PDE1}} + \omega_{\text{PDE3}} \mathcal{L}_{\text{PDE3}} + \omega_{\text{IC1}} \mathcal{L}_{\text{IC1}} + \omega_{\text{IC3}} \mathcal{L}_{\text{IC3}}, \quad (\text{A1})$$

where

$$\mathcal{L}_{\text{PDE1}} = \frac{1}{N_{\text{PDE1}}} \sum_{i=1}^{N_{\text{PDE1}}} \left| \frac{ds_1(t)}{dt} + k_1 s_1(t) - \tilde{k}_3 \tilde{s}_2(t) s_3(t) \right|^2, \quad (\text{A2})$$

$$\mathcal{L}_{\text{PDE3}} = \frac{1}{N_{\text{PDE3}}} \sum_{i=1}^{N_{\text{PDE3}}} \left| \frac{ds_3(t)}{dt} - \tilde{k}_2 \tilde{s}_2(t) \right|^2, \quad (\text{A3})$$

$$\mathcal{L}_{\text{IC1}} = \frac{1}{N_{\text{IC1}}} \sum_{i=1}^{N_{\text{IC1}}} |s_1(0) - 1|^2, \quad (\text{A4})$$

$$\mathcal{L}_{\text{IC3}} = \frac{1}{N_{\text{IC3}}} \sum_{i=1}^{N_{\text{IC3}}} |s_3(0)|^2. \quad (\text{A5})$$

Here, the total loss combines the residual terms from equations (A2) and (A3) ($\mathcal{L}_{\text{PDE1}}$ and $\mathcal{L}_{\text{PDE3}}$), and initial condition terms for s_1 and s_3 ($\mathcal{L}_{\text{IC1}}$ and $\mathcal{L}_{\text{IC3}}$). ω_{PDE1} , ω_{PDE3} , ω_{IC1} , ω_{IC3} , and N_{PDE1} , N_{PDE3} , N_{IC1} , N_{IC3} are weights and number of sampling points in these four terms, respectively.

Similarly, in Case (IIIA), we directly predict variables ($s_1, \tilde{s}_2$) and utilize the conservation law ($s_3(t) = 1 - s_1(t) - \beta\tilde{s}_2(t)$) to derive the indirect variable s_3 .

Specifically, Case (IIIA) excludes the governing equations and initial conditions of the indirect variable s_3 , and applies constraints solely to the directly predicted variables ($s_1, \tilde{s}_2$).

In Case (IIIA), the total loss function $\mathcal{L}(\theta)$ comprises the following components:

$$\mathcal{L}(\theta) = \omega_{\text{PDE1}} \mathcal{L}_{\text{PDE1}} + \omega_{\text{PDE2}} \mathcal{L}_{\text{PDE2}} + \omega_{\text{IC1}} \mathcal{L}_{\text{IC1}} + \omega_{\text{IC2}} \mathcal{L}_{\text{IC2}}, \quad (\text{A6})$$

where

$$\mathcal{L}_{\text{PDE1}} = \frac{1}{N_{\text{PDE1}}} \sum_{i=1}^{N_{\text{PDE1}}} \left| \frac{ds_1(t)}{dt} + k_1 s_1(t) - \tilde{k}_3 \tilde{s}_2(t) s_3(t) \right|^2, \quad (\text{A7})$$

$$\begin{aligned} \mathcal{L}_{\text{PDE2}} = \frac{1}{N_{\text{PDE2}}} \sum_{i=1}^{N_{\text{PDE2}}} & \left| \beta \frac{d\tilde{s}_2(t)}{dt} - k_1 s_1(t) \right. \\ & \left. + \tilde{k}_2 \tilde{s}_2^2(t) + \tilde{k}_3 \tilde{s}_2(t) s_3(t) \right|^2, \end{aligned} \quad (\text{A8})$$

$$\mathcal{L}_{IC1} = \frac{1}{N_{IC1}} \sum_{i=1}^{N_{IC1}} |s_1(0) - 1|^2, \quad (\text{A9})$$

$$\mathcal{L}_{IC2} = \frac{1}{N_{IC2}} \sum_{i=1}^{N_{IC2}} |\tilde{s}_2(0)|^2. \quad (\text{A10})$$

Here, the total loss combines the residual terms from equations (A7) and (A8) ($\mathcal{L}_{PDE1}$ and $\mathcal{L}_{PDE2}$), and initial condition terms for s_1 and $\tilde{s}_2$ ($\mathcal{L}_{IC1}$ and $\mathcal{L}_{IC2}$). The weights and the number of sampling points in these four terms are ω_{PDE1} , ω_{PDE2} , ω_{IC1} , ω_{IC2} , and N_{PDE1} , N_{PDE2} , N_{IC1} , N_{IC2} , respectively.

Appendix B. The loss function of Case (IIA') and Case (IIIA')

In section 3.3.1, we introduced another pure exact constraint form of Case (IB) (indirectly computing s_1). Here, we propose an alternative scenario, i.e., Case (IIB), which retains all terms associated with the indirect variable ($\tilde{s}_2$) during the training procedure and incorporate the conservation law ($\beta\tilde{s}_2(t) = 1 - s_1(t) - s_3(t)$) with predictions of direct variables (s_1, s_3) to derive the indirect variable ($\tilde{s}_2$) during the output stage.

In Case (IIA'), the total loss function $\mathcal{L}(\theta)$ comprises the following components:

$$\mathcal{L}(\theta) = \sum_{i=1}^3 \omega_{PDEi} \mathcal{L}_{PDEi} + \sum_{i=1}^3 \omega_{ICi} \mathcal{L}_{ICi}. \quad (\text{B1})$$

Similarly, in Case (IIIA'), we retain all terms associated with the indirect variable (s_3) during the training procedure and incorporate the conservation law ($s_3(t) = 1 - s_1(t) - \beta\tilde{s}_2(t)$) with predictions of direct variables ($s_1, \tilde{s}_2$) to derive the indirect variable (s_3) during the output stage.

In Case (IIIA'), the total loss function $\mathcal{L}(\theta)$ comprises the following components:

$$\mathcal{L}(\theta) = \sum_{i=1}^3 \omega_{PDEi} \mathcal{L}_{PDEi} + \sum_{i=1}^3 \omega_{ICi} \mathcal{L}_{ICi}. \quad (\text{B2})$$

Appendix C. The loss function of Case (IIB) and Case (IIIB)

In section 3.3.2, we introduced a hybrid constraints strategy in Case (IB) (indirectly computing s_1). Here, we propose an alternative scenario, i.e., Case (IIB), which retains all terms associated with the indirect variable ($\tilde{s}_2$) during the training procedure and incorporate the conservation law ($\beta\tilde{s}_2(t) = 1 - s_1(t) - s_3(t)$) with predictions of direct variables (s_1, s_3) to derive the indirect variable ($\tilde{s}_2$) during the output stage. Meanwhile, it simultaneously incorporates the conservation law term (see equation (28)) into the loss function.

In Case (IIB), the total loss function $\mathcal{L}(\theta)$ comprises the following components:

$$\mathcal{L}(\theta) = \sum_{i=1}^3 \omega_{PDEi} \mathcal{L}_{PDEi} + \sum_{i=1}^3 \omega_{ICi} \mathcal{L}_{ICi} + \omega_{CON} \mathcal{L}_{CON}. \quad (\text{C1})$$

Similarly, in Case (IIIB), we retain all terms associated with the indirect variable (s_3) during the training procedure and incorporates the conservation law ($s_3(t) = 1 - s_1(t) - \beta\tilde{s}_2(t)$) with predictions of direct variables ($s_1, \tilde{s}_2$) to derive the indirect variable (s_3) during the output stage. Meanwhile, it simultaneously incorporates the conservation law term (see equation (28)) into the loss function.

In Case (IIIB), the total loss function $\mathcal{L}(\theta)$ comprises the following components:

$$\mathcal{L}(\theta) = \sum_{i=1}^3 \omega_{PDEi} \mathcal{L}_{PDEi} + \sum_{i=1}^3 \omega_{ICi} \mathcal{L}_{ICi} + \omega_{CON} \mathcal{L}_{CON}. \quad (\text{C2})$$

Appendix D. Impact of the hyperparameters on EC-PINN's performance (Case (IA))

For clarity in the analysis, all discussions are presented using Case (IA) as an example. In Case (IA), s_1 as an indirect variable and does not participate in the training of the PINN. Instead, it is derived from $\tilde{s}_2$ and s_3 using the conservation law during the output stage. Therefore, there are two equations to solve in Case (IA). In other words, we set two branch networks. For simplicity, these two branch networks share the same number of layers.

Next, we discuss uncertainties involved in the hyperparameters, such as the number of layers in the shared network. In this work, the network architecture consists of shared layers and branch layers. To proceed, we take Case (IA) as an example to investigate the impact of the number of shared network layers on the PINN's

Table 4. The settings of hyperparameters.

Hyperparameters	Settings
Normalization parameter	$\beta = 10^{-4}$
Sampling points	1000
Number of layers	Shared network: 7 layers; Branch networks: 5 layers
Number of neurons	100
Activation function	Tanh
Weight	$\omega_{\text{PDE}i} = \omega_{\text{IC}i} = 1$ ($i=1,2,3$)
Optimizer	SOAP and L-BFGS
Learning rates	SOAP: 10^{-3} ; L-BFGS: 1
Iteration steps	SOAP: 5000; L-BFGS: 2000

performance. Here, the total number of neural network is fixed at 12. Based on this, we will evaluate the impact of the number of shared network layers, as detailed below:

As shown in table 3, the standard DNN architecture (i.e., the number of shared network layers is 12) cannot solve the stiff chemical kinetics with high accuracy. For instance, the relative error of $\tilde{s}_2$ is about 17.6%. However, when the shared-branch network is introduced (i.e., the number of shared network layers is less than 12), all variables can be solved effectively under different combinations. Nevertheless, the calculation accuracy and computational costs vary significantly across different combinations. Even though, there is currently no suitable principle to guide us on how to select the number of layers in a shared network. This fact may indicate that the current method has certain limitations.

It should be noted that, all hyperparameters listed in table 4 can influence the performance of PINN. However, a guiding principle for selecting hyperparameters is still lacking. Instead, we typically rely on empirical rules. In this work, we primarily examine how the strict conservation law constraint affects the performance of PINN in solving stiff chemical kinetics. For simplicity, we empirically select the hyperparameters used in this work, as shown in table 4. In the future, we will try to find a theory-based principle for selecting hyperparameters and conduct sensitivity analysis of these parameters.

Appendix E. Results of EC-PINN under different random seeds (Case (IA))

In order to improve the reliability of our EC-PINN, we take Case (IA) as an example and solve the stiff chemical kinetics problem with EC-PINN based on four different random seeds. The mean relative error and standard deviation are shown in table 5.

From table 5, the accuracy of s_1 , $\tilde{s}_2$, and s_3 under different random seeds is comparable. Furthermore, the standard deviations in these four numerical experiments are much less than 1. Consequently, different random initializations have minimal impact on the performance of EC-PINN, which demonstrates the stability and reproducibility of our framework.

Table 5. Mean and standard deviation of relative errors of case (IA) with four different random seeds.

	Relative error of s_1	Relative error of $\tilde{s}_2$	Relative error of s_3
1	8.81×10^{-5}	8.45×10^{-3}	4.29×10^{-3}
2	9.34×10^{-5}	7.33×10^{-3}	4.54×10^{-3}
3	5.65×10^{-6}	6.74×10^{-3}	2.35×10^{-4}
4	2.90×10^{-5}	4.79×10^{-3}	1.40×10^{-3}
Mean	5.40×10^{-5}	6.83×10^{-3}	2.62×10^{-3}
Standard Deviation	4.35×10^{-5}	1.53×10^{-3}	2.13×10^{-3}

Appendix F. Symbols

To clarify, we present the table to illustrate symbols. The definitions of symbols are shown in table 6:

Table 6. The definitions of symbols.

Symbols	definitions
t	Time
x	Space
$\mathcal{N}$	Linear or nonlinear operator
$\mathcal{B}$	Boundary condition operator
Ω	Domain
R^D	D-dimensional Euclidean space
$\mathcal{L}$	Loss function
ω	Weight of loss function
θ	Trainable parameters of the network
L_2	L_2 norm (Euclidean norm)
s	Chemical species concentration
k	Reaction rate constant
β	Normalization parameter
E	Conserved quantity
u	Ground truth of the governing equations
u_θ	Predicted solution of the governing equations
N_t	The number of sampling points

Author contributions

Hanmin Fang

Data curation (equal), Formal analysis (equal), Investigation (equal), Methodology (equal), Visualization (equal), Writing – original draft (equal)

Na Wang

Supervision (equal), Validation (equal), Writing – review & editing (equal)

Haolong Li  0000-0002-7050-1295

Formal analysis (equal), Investigation (equal), Writing – review & editing (equal)

References

- [1] Huang Y and Seinfeld J H 2022 A neural network-assisted euler integrator for stiff kinetics in atmospheric chemistry *Environ. Sci. Technol.* **56** 4676–85
- [2] Chaudhuri J H and Chatterjee D 2024 Modelling of chemical kinetics in the presence of hydrodynamic cavitation for wastewater treatment applications *Chem. Eng. Sci.* **295** 120167
- [3] Gou X, Sun W, Chen Z and Ju Y 2010 A dynamic multi-timescale method for combustion modeling with detailed and reduced chemical kinetic mechanisms *Combust. Flame* **157** 1111–21
- [4] Robertson H H 1966 The solution of a set of reaction rate equations *Numerical Analysis: An Introduction* **178** 182–31
- [5] Voss D A and Khaliq A-Q M 2001 Parallel rosenbrock methods for chemical systems *Comput. Chem. (Oxford)* **25** 101–7
- [6] Sandu A and Miehe P 2010 Forward, tangent linear, and adjoint runge-kutta methods for stiff chemical kinetic simulations *Int. J. Comput. Math.* **87** 2458–79
- [7] Khalsaraei M M, Shokri A and Molayi M 2020 The new class of multistep multiderivative hybrid methods for the numerical solution of chemical stiff systems of first order ivps *J. Math. Chem.* **58** 1987–2012
- [8] Raissi M, Perdikaris P and Karniadakis G E 2019 Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations *J. Comput. Phys.* **378** 686–707
- [9] Tanios R 2021 Physics informed neural networks in computational finance: high dimensional forward and inverse option pricing *Master's thesis, ETH Zurich*
- [10] Sahli Costabal F, Pezzuto S and Perdikaris P 2024 δ -pinns: Physics-informed neural networks on complex geometries *Eng. Appl. Artif. Intell.* **127** 107324
- [11] He A, Shi J, Chen J and Fang H 2024 The data-driven solutions and inverse problems of some nonlinear diffusion convection-reaction equations based on physics-informed neural network *Phys. Scr.* **99** 116001
- [12] Bararnia H and Esmailpour M 2022 On the application of physics informed neural networks (pinn) to solve boundary layer thermal-fluid problems *Int. Commun. Heat Mass Transfer* **132** 105890
- [13] Wu W, Daneker M, Turner K T, Jolley M A and Lu L 2025 Identifying heterogeneous micromechanical properties of biological tissues via physics-informed neural networks *Small Methods* **9** 2400620

- [14] Han S, Stelz L, Stoecker H, Wang L and Zhou K 2024 Approaching epidemiological dynamics of covid-19 with physics-informed neural networks *J. Franklin Inst.* **361** 106671
- [15] Zhang E, Dao M, Karniadakis G E and Suresh S 2022 Analyses of internal structures and defects in materials using physics-informed neural networks *Science Advances* **8** eabk0644
- [16] Lee S and Popovics J 2022 Applications of physics-informed neural networks for property characterization of complex materials *RILEM Technical Letters* **7** 178–88
- [17] Wang S, Teng Y and Perdikaris P 2021 Understanding and mitigating gradient flow pathologies in physics-informed neural networks *SIAM J. Sci. Comput.* **43** A3055–81
- [18] Wang Z, Hu Z, Yang M, Dong Y, Huang W and Ren H 2025 Solving partial differential equations based on preconditioning-pretraining physics-informed neural network *Comput. Phys. Commun.* **312** 109601
- [19] Ji W, Qiu W, Shi Z, Pan S and Deng S 2021 Stiff-pinn: physics-informed neural network for stiff chemical kinetics *The Journal of Physical Chemistry A* **125** 8098–106
- [20] Weng Y and Zhou D 2022 Multiscale physics-informed neural networks for stiff chemical kinetics *The Journal of Physical Chemistry A* **126** 8534–43
- [21] Fang H, Huang W and Wang Z 2025 Long-time simulation of stiff chemical kinetics using conservation-constrained physics-informed neural network *Chin. J. Space Sci.* **45** 277–87
- [22] Ko S and Park S 2025 Vs-pinn: a fast and efficient training of physics-informed neural networks using variable-scaling methods for solving pdes with stiff behavior *J. Comput. Phys.* **529** 113860
- [23] Oluwasakin E O, Khaliq A Q M and Furati K M 2025 Learning time-varying parameters of stiff dynamical systems using physics-informed transfer neural network *Math. Comput. Simul.* (<https://doi.org/10.1016/j.matcom.2025.04.038>)
- [24] Deng F, Wu J, Yang Y, Li J, Xie X, Jiang J and Zhu X 2025 Improved physics-informed neural network for reactive transport modeling of groundwater arsenic enrichment *Science China Earth Sciences* **68** 2781–96
- [25] Nasir S, Sahibzada S and Malik F S 2025 Physics-informed neural networks for predicting combustion dynamics in rocket engine chambers *International Journal of Innovative Research in Computer Science and Technology* **13** 41–4
- [26] Sivalingam S M and Govindaraj V 2024 Physics informed neural network based scheme and its error analysis for ψ -caputo type fractional differential equations *Phys. Scr.* **99** 096002
- [27] Jiahao S, Wenbo C and Weiwei Z 2023 Fd-pinn: Frequency domain physics-informed neural network *Chinese Journal of Theoretical and Applied Mechanics* **55** 1195–205
- [28] Wang S, Sankaran S, Wang H and Perdikaris P 2023 An expert's guide to training physics-informed neural networks arXiv:2308.08468
- [29] Li X, Liu Y and Liu Z 2023 Physics-informed neural network based on a new adaptive gradient descent algorithm for solving partial differential equations of flow problems *Phys. Fluids* **35** 063608
- [30] Guo J, Yao Y, Wang H and Gu T 2023 Pre-training strategy for solving evolution equations based on physics-informed neural networks *J. Comput. Phys.* **489** 112258
- [31] Vyas N, Morwani D, Zhao R, Kwun M, Shapira I, Brandfonbrener D, Janson L and Kakade S 2024 Soap: Improving and stabilizing shampoo using adam arXiv:2409.11321
- [32] Wang S, Kumar Bhartari A, Li B and Perdikaris P 2025 Gradient alignment in physics-informed neural networks: A second-order optimization perspective arXiv:2502.00604
- [33] Byrd R H, Lu P, Nocedal J and Zhu C 1995 A limited memory algorithm for bound constrained optimization *SIAM J. Sci. Comput.* **16** 1190–208
- [34] Liu D C and Nocedal J 1989 On the limited memory bfgs method for large scale optimization *Math. Program.* **45** 503–28
- [35] Xiaoguo P, Kai W and Weixin D 2024 Accelerating convergence algorithm for physics-informed neural networks based on ntk theory and modified causality *Chinese Journal of Theoretical and Applied Mechanics* **56** 1943–58
- [36] Sivamohan S, Sridhar S S and Krishnaveni S 2021 An effective recurrent neural network (rnn) based intrusion detection via bi-directional long short-term memory 2021 *International Conference on Intelligent Technologies (CONIT)* 1–5 (IEEE)
- [37] Geneva N and Zabarás N 2022 Transformers for modeling physical systems *Neural Netw.* **146** 272–89
- [38] Yang S D, Ali Z A and Wong B M 2023 Fluid-gpt (fast learning to understand and investigate dynamics with a generative pre-trained transformer): Efficient predictions of particle trajectories and erosion *Ind. Eng. Chem. Res.* **62** 15278–89
- [39] Duranoğlu D, Altın E S and Küçük İ 2024 Optimization of adaptive neuro-fuzzy inference system (anfis) parameters via box-behnken experimental design approach: The prediction of chromium adsorption *Heliyon* **10** e25813
- [40] Kardan N, Motamadnia A, Ghaderpour E and Mazzanti P 2025 Employing advanced optimization algorithms to forecast the depth of scouring downstream of culvert boxes *Results in Engineering* 107482
- [41] De Florio M, Schiassi E and Furfaro R 2022 Physics-informed neural networks and functional interpolation for stiff chemical kinetics *Chaos* **32**
- [42] Kim G and Heo G 2023 Solving partial differential equation for atmospheric dispersion of radioactive material using physics-informed neural network *Nuclear Engineering and Technology* **55** 2305–14
- [43] Rashid S, Abbas H, Ali I, Khalid A, Tesgera Agama F and Aldien M S 2025 Evaluating physics informed neural networks for water contamination risk prediction and environmental sustainability *Sci. Rep.* (<https://doi.org/10.1038/s41598-025-30196-x>)